

天津大学

网络安全学院

《软件安全》实验报告

实验一：PE 文件代码注入实验

姓名：____安孟喆_____

学 号：__3024244059_____

一、实验目的

通过本实验，预期达到以下实验目的：

- 熟悉 Windows PE 文件的基本结构，理解入口点、镜像基址、节表、文件对齐与内存对齐等概念。
- 复习汇编语言常见指令，能够分析 push、call、jmp 等指令在程序执行流程中的作用。
- 学习使用 LordPE 查看、编辑并保存 PE 文件头部信息。
- 熟练使用 OllyICE/OllyDbg 对目标程序进行动态调试、地址定位和代码修改。

二、实验环境

- 操作系统：Windows XP Professional 虚拟机。
- 目标程序：Windows 自带扫雷程序 winmine.exe。
- 主要工具：LordPE、OllyICE/OllyDbg、Windows 资源管理器。
- 实验目标：在不破坏原有功能的前提下修改 winmine.exe 的 PE 入口点，使程序启动时先弹出自定义提示框，然后跳回原入口点继续执行。

三、实验原理

PE 文件被 Windows 装载时，系统会根据 Optional Header 中的 AddressOfEntryPoint 定位程序入口。该入口点通常以 RVA 形式保存，实际运行时地址 $VA = ImageBase + RVA$ 。本实验通过在目标文件可利用的空白区域写入一段实验性代码，再把 PE 入口点改为新代码的 RVA，从而让程序启动时先执行注入代码。

注入代码的核心逻辑是调用 MessageBoxA 显示“PE INJECT / You are Injected!”提示框。提示框执行完后，代码必须使用 jmp 指令跳回原入口点，否则程序原有启动逻辑无法继续执行。

本实验中记录到的关键地址关系如下：原入口点 RVA 为 00003E21，对应运行时 VA 为 01003E21；用于启动注入逻辑的新入口点 RVA 为 00004AAE；字符串区域位于 01004A8F 附近，注入代码末尾跳回 01003E21。

四、实验步骤

1. 打开实验工具并定位目标程序

在 Windows XP 虚拟机中进入工具目录，打开 LordPE。实验目录中可以看到 OllyICE、LordPE、winmine.exe 等工具和目标文件。

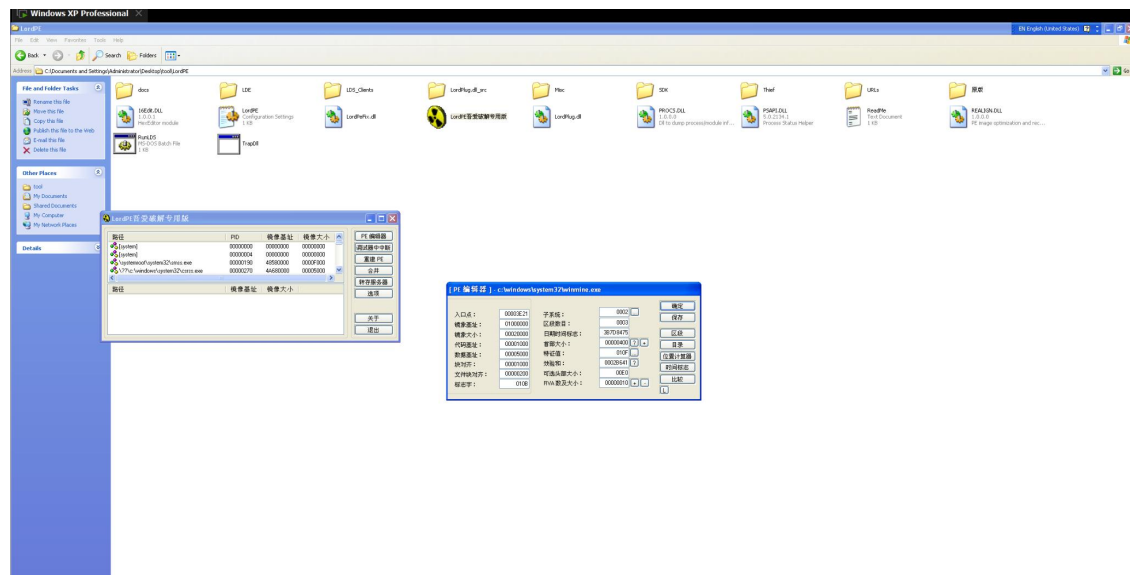


图 1 打开 LordPE 并准备查看目标程序

2. 查看 PE 文件原始入口点

使用 LordPE 的 PE 编辑器打开 winmine.exe，记录 PE 头部信息。截图中可见镜像基址为 01000000，代码基址为 00001000，数据基址为 00005000，区段数为 0003。原始入口点为 00003E21，因此程序正常启动时会从 $VA=01000000+00003E21=01003E21$ 处开始执行。

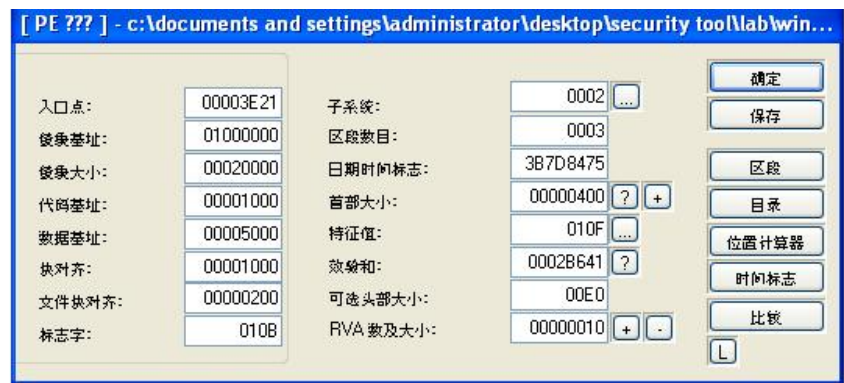


图 2 使用 LordPE 查看原入口点 $RVA=00003E21$

3. 在 OllyICE 中定位代码洞并写入字符串

用 OllyICE 打开 winmine.exe 后，在文件尾部或节尾部可利用的 00 字节区域写入提示框标题和内容字符串。截图中写入了 ASCII 字符串“PE INJECT”和“You are Injected!”，用于后续 MessageBoxA 调用。

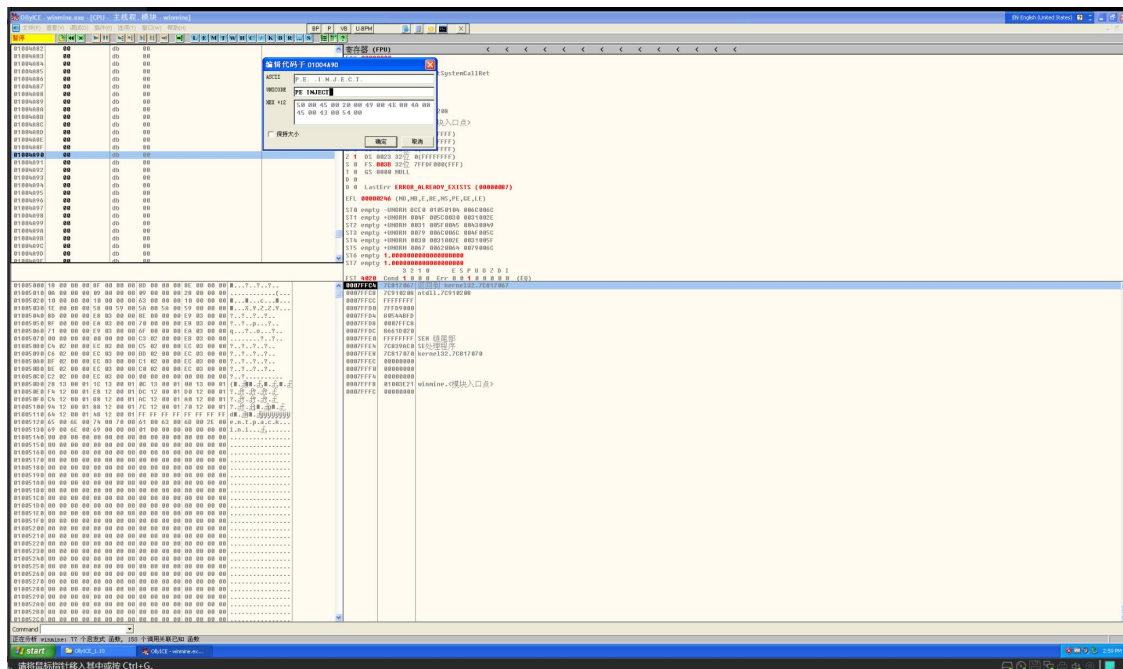


图 3 在 OllyICE 中编辑空白区域并写入提示字符串

4. 写入弹窗调用代码并设置跳转回原入口点

在字符串之后写入汇编代码，依次压入 MessageBoxA 的四个参数，然后调用 USER32.MessageBoxA。弹窗执行结束后，使用 jmp 指令跳转回原入口点 01003E21，使扫描雷程序继续按原流程执行。

```
01004A8F    ASCII "PE INJECT"
01004A99    ASCII "You are Injected!"
01004AAE    PUSH 0
01004AB0    PUSH 01004A99          ; lpText
01004AB5    PUSH 01004A8F          ; lpCaption
01004ABA    PUSH 0
01004ABC    CALL USER32.MessageBoxA
01004AC1    JMP 01003E21           ; 跳回原入口点
```

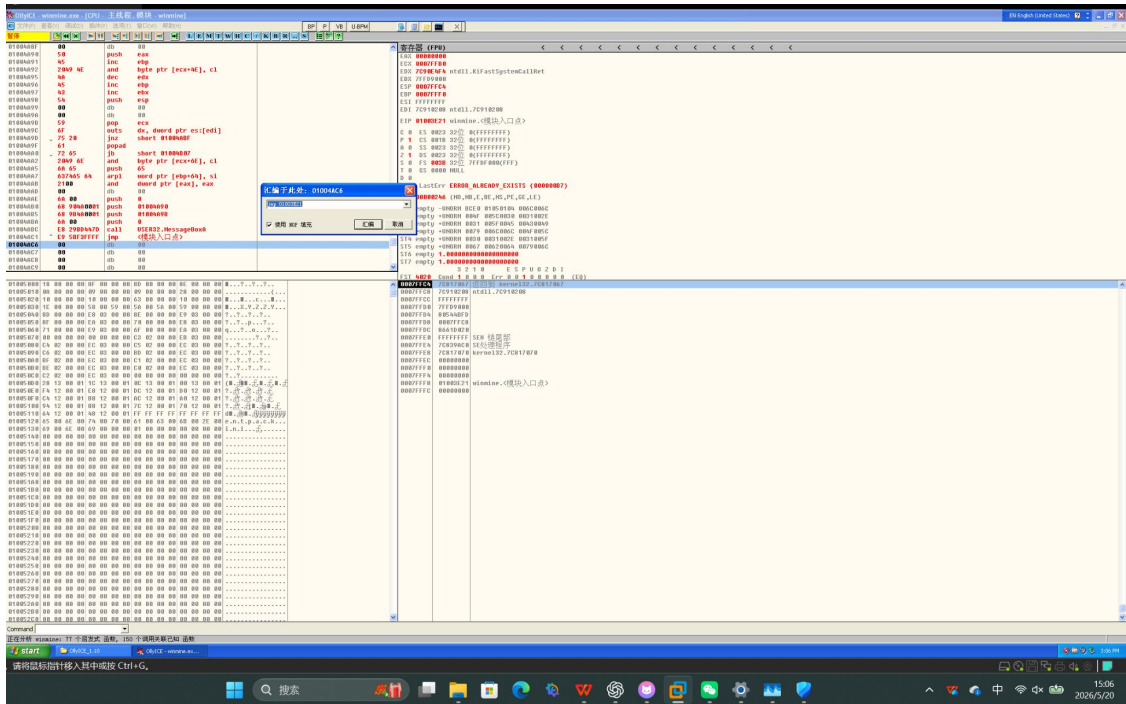


图 4 汇编 jmp 指令，将执行流程跳回原入口点 01003E21

			byte ptr [ecx+4E], c1	
			edx	
01004A96	45	inc	ebp	
01004A97	43	inc	ebx	
01004A98	54	push	esp	
01004A99	00	db	00	
01004A9A	00	db	00	
01004A9B	59	pop	ecx	
01004A9C	6F	outs	dx, dword ptr es:[edi]	
01004A9D	75 20	jnz	short 01004ABF	
01004A9E	61	popad		
01004A9F	72 65	jb	short 01004B07	
01004AA0	2049 6E	and	byte ptr [ecx+6E], c1	
01004AA1	6A 65	push	65	
01004AA2	637465 64	arpl	word ptr [ebp+64], si	
01004AA3	2100	and	dword ptr [eax], eax	
01004AA4	00	db	00	
01004AA5	6A 00	push	0	
01004AA6	68 9B4A0001	push	01004A90	
01004AA7	68 9B4A0001	push	01004A9B	
01004AA8	6A 00	push	0	
01004AA9	E8 29B0447D	call	USER32.MessageBoxA	
01004AAC	E9 5BF3FFFF	jmp	<模块入口点>	
01004AC1	00	db	00	
01004AC2	00	db	00	
01004AC3	00	db	00	
01004AC4	00	db	00	
01004AC5	00	db	00	
01004AC6	00	db	00	
01004AC7	00	db	00	
01004AC8	00	db	00	
01004AC9	00	db	00	
01004ACA	00	db	00	
01004ACB	00	db	00	

图 5 注入区域中可见字符串、MessageBoxA 调用和返回原入口点的跳转

5. 修改 PE 入口点并保存

返回 LordPE 的 PE 编辑器，把 AddressOfEntryPoint 从原来的 00003E21 修改为注入代码开始处对应的 RVA 00004AAE。注意这里填写的是 RVA，不是 01004AAE 这种运行时 VA。修改完成后保存 PE 文件。

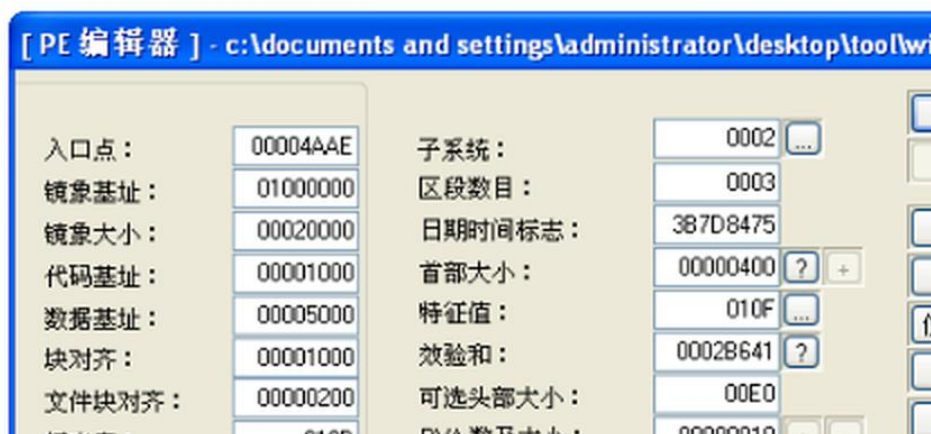


图 6 将 PE 入口点修改为新代码 RVA=00004AAE

6. 运行程序验证实验结果

双击运行修改后的 winmine.exe。程序启动时首先弹出标题为“PE INJECT”、内容为“You are Injected!”的对话框，说明入口点已经成功跳转到注入代码。点击 OK 后，程序继续跳回原入口点，原程序功能不被破坏。



图 7 修改后运行 winmine.exe，成功弹出自定义提示框

五、实验结果分析

从运行结果看，目标程序启动后能够先执行新增代码并弹出自定义提示框，随后仍能回到原入口点继续运行，说明本次 PE 代码注入流程成功。该结果验证了 PE 入口点控制程序初始执行位置的作用，也验证了“新入口点执行自定义代码—跳回原入口点”的流程能够保持原程序功能。

本实验中最关键的是区分 RVA 与 VA。LordPE 中入口点字段应填写 RVA 00004AAE；而 OllyICE 调试窗口中看到的是运行时虚拟地址 01004AAE。二者之间相差 ImageBase=01000000。若把 VA 误填到 PE 头入口点字段中，程序装载后将无法正确定位入口。

六、实验中遇到的问题及解决办法

1. 原入口点和新入口点容易混淆。解决方法是先在 LordPE 中记录原入口点 00003E21，再在 OllyICE 中计算对应 VA=01003E21；修改入口点时只填写新代码的 RVA 00004AAE。
2. 注入代码执行后如果不跳回原入口点，程序会无法继续运行。解决方法是在调用 MessageBoxA 后补充 jmp 01003E21，让程序返回原来的启动逻辑。
3. 调试器中同时存在文件偏移、RVA 和 VA，直接照抄地址容易出错。解决方法是结合 ImageBase 和节信息判断地址含义，并在每次修改后重新运行程序验证。
4. 部分老工具界面可能出现中文乱码或按钮含义不清。解决方法是根据窗口位置和功能名称判断操作，例如“PE 编辑器”“保存”“确定”等；必要时使用英文路径和英文工具界面，避免路径编码影响工具运行。

七、心得体会

通过本次实验，我对 PE 文件结构有了更直观的理解。以前只是知道可执行文件有入口点和节表，但没有真正体会入口点如何影响程序的第一条执行指令。本次通过 LordPE 修改 AddressOfEntryPoint，再用 OllyICE 写入弹窗代码和跳转代码，能够清楚看到程序从新入口点开始执行，再返回原入口点继续运行。

实验还让我加深了对 RVA、VA、ImageBase 三者关系的理解。PE 文件中的入口点不是调试器里看到的完整内存地址，而是相对镜像基址的偏移。实际操作中如果混淆这几个地址，程序就可能无法启动。

此外，本实验也说明代码注入不仅是简单写入几条汇编指令，还必须考虑调用参数、地址计算、程序控制流恢复和文件保存。通过动手完成整个过程，我对 PE 文件查看、编辑和调试工具的配合使用更加熟悉。